DiceKriging vs RobustGaSP

Gu (2019)

Prediction

We now provide an example in which the input has one dimension, ranging from r0, 10s (Higdon and others (2002)). Estimation of the range parameters using the RobustGaSP package can be done through the following code:

```
library(RobustGaSP)
library(lhs)
set.seed(1)
input <- 10 * maximinLHS(n=15, k=1)
output <- higdon.1.data(input)
model <- rgasp(design = input, response = output)
```

```
## The upper bounds of the range parameters are 395.6186
## The initial values of range parameters are 7.912373
## Start of the optimization  1  :
## The number of iterations is  10
##  The value of the  marginal posterior  function is  2.81014
##  Optimized range parameters are 1.768226
##  Optimized nugget parameter is 0
##  Convergence:  TRUE
## The initial values of range parameters are 0.08251576
## Start of the optimization  2  :
## The number of iterations is  12
##  The value of the  marginal posterior  function is  2.81014
##  Optimized range parameters are 1.768226
##  Optimized nugget parameter is 0
##  Convergence:  TRUE
```

```
model
```

```
##
## Call:
## rgasp(design = input, response = output)
## Mean parameters:  0.01184582
## Variance parameter:  0.5697197
## Range parameters:  1.768226
## Noise parameter:  0
```

The fourth line of the code generates 15 LH samples at $[0, 10]$ through the `maximinLHS` function of the `lhs` package (Carnell (2016)). The function `higdon.1.data` is provided within the `RobustGaSP` package which

has the form $y(x) = \sin(2\pi x/10) + 0.2\sin(2\pi x/2.5)$. The third line fits a GaSP model with the robust parameter estimation by marginal posterior modes.

The plot function in RobustGaSP package implements the leave-one-out cross validation for a "rgasp" class after the GaSP model is built (see Figure 1 for its output):

```
plot(model)
```

The prediction at a set of input points can be done by the following code:

```
testing_input <- as.matrix(seq(0, 10, 1/50))
model.predict<-predict(model, testing_input)
names(model.predict)
```

```
## [1] "mean"    "lower95" "upper95" "sd"
```

The predict function generates a list containing the predictive mean, lower and upper 95% quantiles and the predictive standard deviation, at each test point $\mathbf{x}^*$. The prediction and the real outputs are plotted in Figure 2; produced by the following code:

```
testing_output <- higdon.1.data(testing_input)
plot(testing_input, model.predict$mean,type='l', col='blue',
     xlab='input', ylab='output')
polygon( c(testing_input,rev(testing_input)), c(model.predict$lower95,
                                                rev(model.predict$upper95)), col = "grey80", border = F
lines(testing_input, testing_output)
lines(testing_input,model.predict$mean, type='l', col='blue')
lines(input, output, type='p')
```

It is also possible to sample from the predictive distribution (which is a multivariate $ distribution) using the following code:

```
model.sample <- RobustGaSP::simulate(model, testing_input, num_sample=10)
matplot(testing_input, model.sample, type='l', xlab='input', ylab='output')
lines(input, output,type='p')
```

The plots of 10 posterior predictive samples are shown in Figure 3.

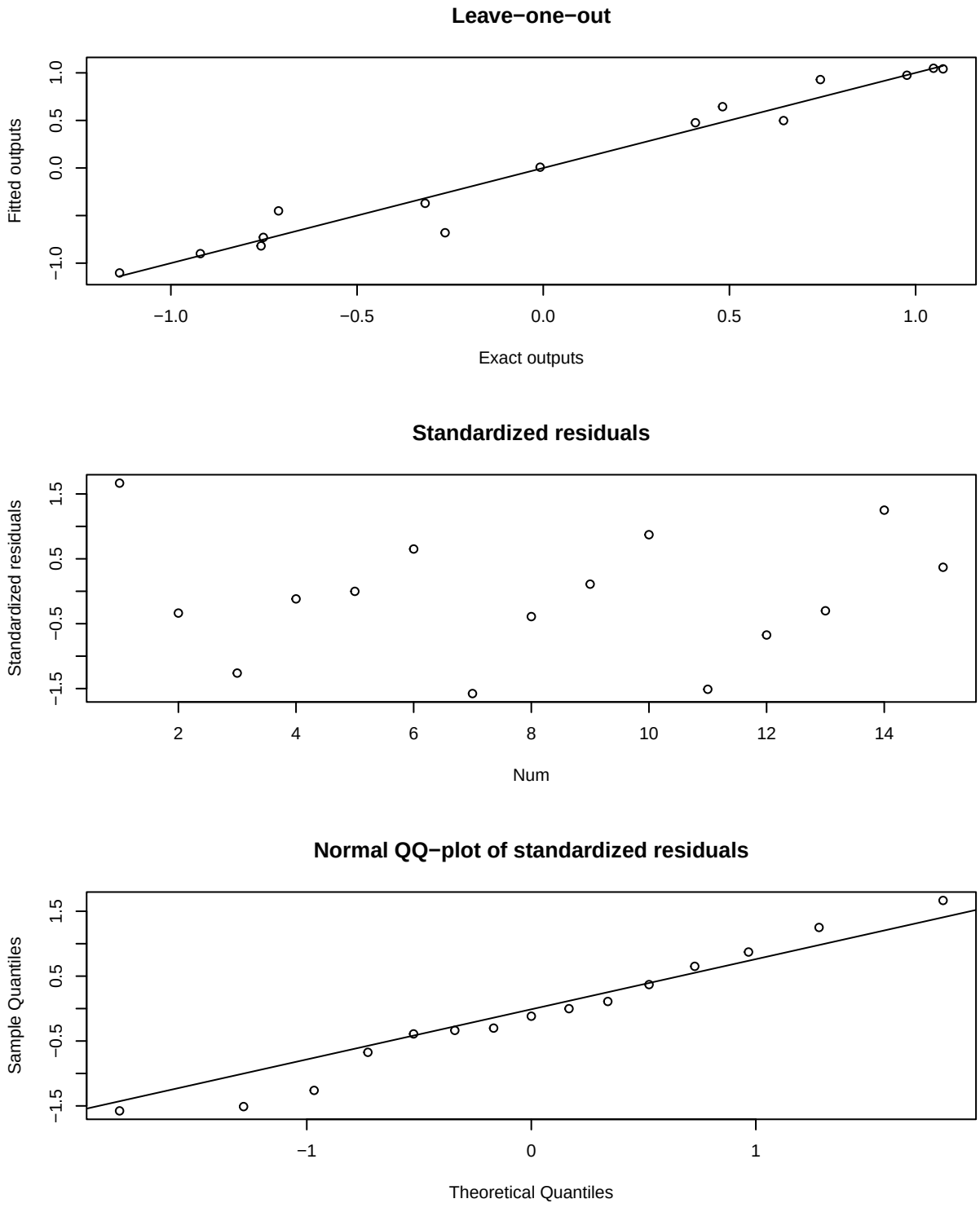

Figure 1: Figure 1

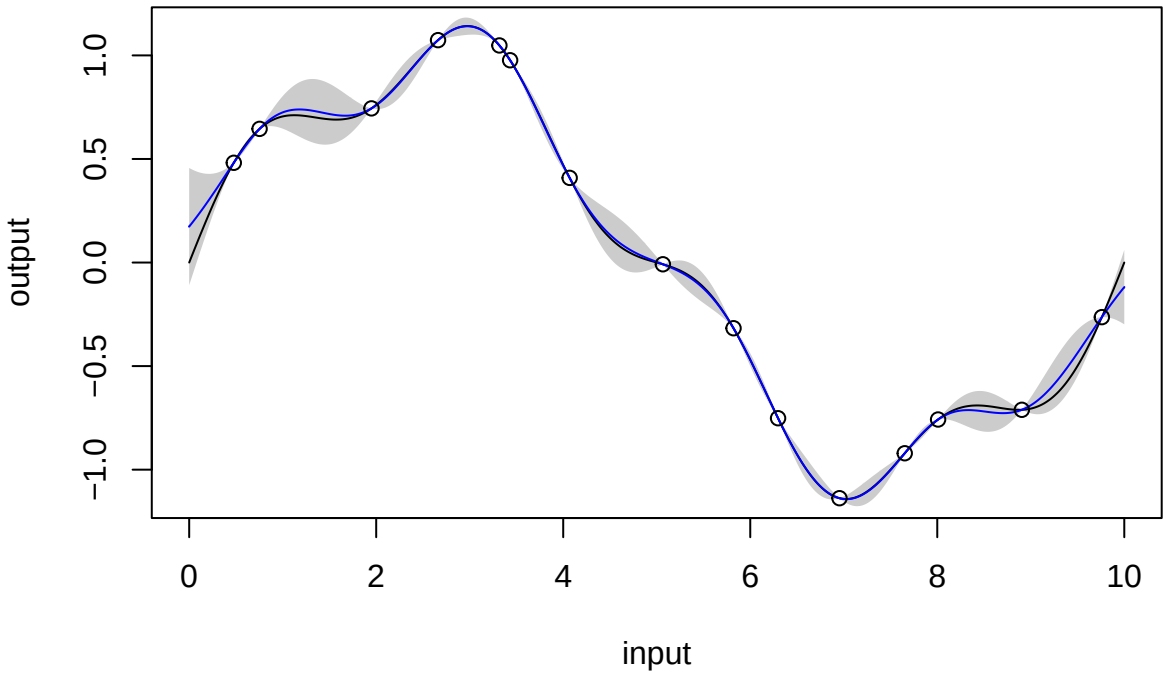

Figure 2: Figure 2

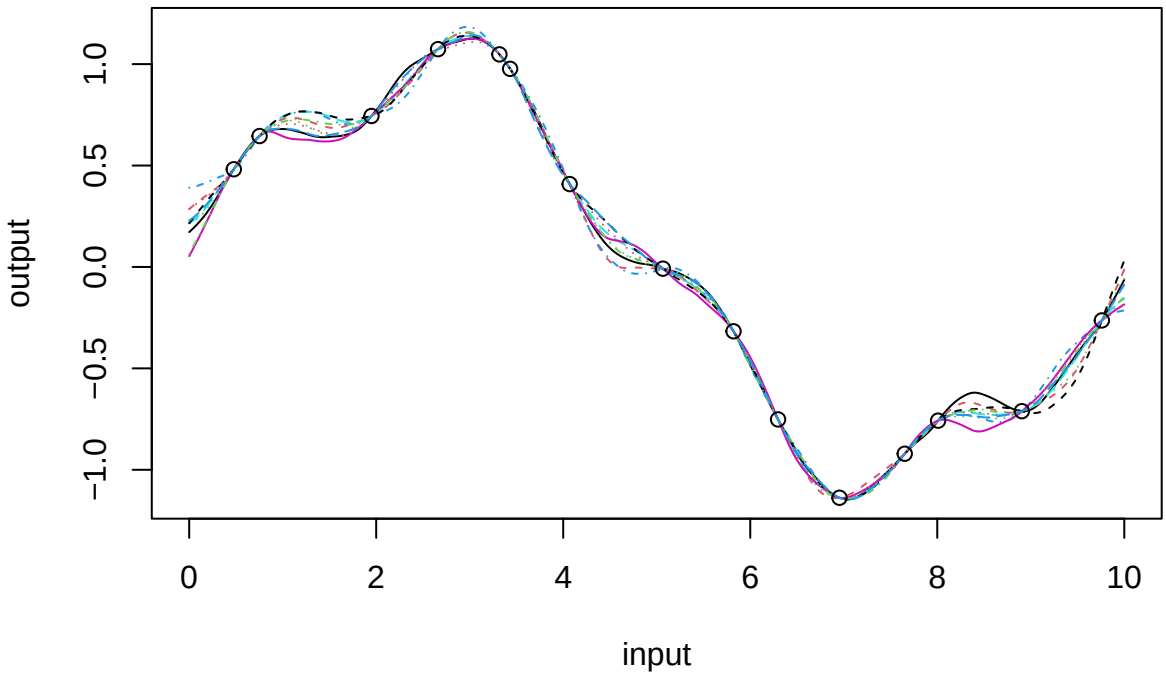

Figure 3: Figure 3

Identification of inert inputs

We use 40 maximin LH samples to find inert inputs at the borehole function through the following code.

```
set.seed(1)
input <- maximinLHS(n=40, k=8) # maximin lhd sample

# rescale the design to the domain of the borehole function
LB <- c(0.05, 100, 63070, 990, 63.1, 700, 1120, 9855)
UB <- c(0.15, 50000, 115600, 1110, 116, 820, 1680, 12045)
range <- UB - LB
for(i in 1:8) {
  input[,i] = LB[i] + range[i] * input[,i]
}
num_obs <- dim(input)[1]
output <- matrix(0,num_obs,1)
for(i in 1:num_obs) {
  output[i] <- borehole(input[i,])
  }
m <- rgasp(design = input, response = output, lower_bound=FALSE)
```

```
## The upper bounds of the range parameters are Inf Inf Inf Inf Inf Inf Inf Inf
## The initial values of range parameters are 0.001059729 0.01999988 0.01999989 0.01995154 0.01988913 0
## Start of the optimization  1  :
## The number of iterations is  28
##  The value of the  marginal posterior  function is  -292.7766
##  Optimized range parameters are 880131429831218195204024206600660042220402224240682228042862080624803
##  Optimized nugget parameter is 0
##  Convergence:  FALSE
## The initial values of range parameters are 0.6233334 310593 329576.8 754.7742 329.4202 740.7992 3462
## Start of the optimization  2  :
## The number of iterations is  67
##  The value of the  marginal posterior  function is  -127.4453
##  Optimized range parameters are 0.1950828 15754792663288 146077093533443 771.5979 41857.69 873.9928 3
##  Optimized nugget parameter is 0
##  Convergence:  TRUE
```

```
P <- findInertInputs(m)
```

```
## The estimated normalized inverse range parameters are : 4.031267 0.00000002487248 0.000000002846522 1
## The inputs  2 3 5 are suspected to be inert inputs
```

Similar to the automatic relevance determination model in neural networks, e.g. MacKay (1996); Neal (1996), and in machine learning, e.g. Tipping (2001); Li et al. (2002), the function `findInertInputs` of the `RobustGaSP` package indicates that the 2nd, 3rd, and 5th inputs are suspected to be inert inputs. Figure 4 presents the plots of the borehole function when varying one input at a time. This analyzes the local sensitivity of an input when having the others fixed. Indeed, the output of the borehole function changes very little when the 2nd, 3rd, and 5th inputs vary.

```
m.subset <- rgasp(design = input[ ,c(1,4,6,7,8)], response = output,
                  nugget.est=TRUE)
```

```
## The upper bounds of the range parameters are 19.34479 23423.98 22990.27 107444.3 416986.6 Inf
## The initial values of range parameters are 0.3868958 468.4796 459.8055 2148.887 8339.732
## Start of the optimization  1  :
## The number of iterations is  45
##  The value of the  marginal posterior  function is  -126.1572
##  Optimized range parameters are 0.1836021 693.1441 777.1468 3167.149 33145.77
##  Optimized nugget parameter is 0.00000000000000003253473
##  Convergence:  TRUE
## The initial values of range parameters are 0.2240224 271.2614 266.2389 1244.26 4828.915
## Start of the optimization  2  :
## The number of iterations is  45
##  The value of the  marginal posterior  function is  -126.1572
##  Optimized range parameters are 0.1836022 693.1457 777.1462 3167.157 33145.8
##  Optimized nugget parameter is 0.00000000000000005056662
##  Convergence:  TRUE
```

```
m.subset
```

```
##
## Call:
## rgasp(design = input[, c(1, 4, 6, 7, 8)], response = output,
##     nugget.est = TRUE)
## Mean parameters:  142.1235
## Variance parameter:  76967.69
## Range parameters:  0.1836021 693.1441 777.1468 3167.149 33145.77
## Noise parameter:  0.0000000002504123
```

To compare the performance of the emulator with and without a noise term, we perform some out-of-sample testing. We build the GaSP emulator by the `RobustGaSP` package and the `DiceKriging` package using the same mean and covariance. In `RobustGaSP`, the parameters in the correlation functions are estimated by marginal posterior modes with the robust parameterisation, while in `DiceKriging`, parameters are estimated by MLE with upper and lower bounds. We first construct these four emulators with the following code

```
m.full <- rgasp(design = input, response = output)
```

```
## The upper bounds of the range parameters are 123.6899 61631867 65398881 149772 65367.81 146999 686995
## The initial values of range parameters are 2.473797 1232637 1307978 2995.441 1307.356 2939.979 13739
## Start of the optimization  1  :
## The number of iterations is  31
##  The value of the  marginal posterior  function is  -127.7353
##  Optimized range parameters are 0.1932037 61631867 65398881 709.2224 31879.72 806.2708 3478.621 33790
##  Optimized nugget parameter is 0
##  Convergence:  TRUE
## The initial values of range parameters are 0.6233334 310593 329576.8 754.7742 329.4202 740.7992 3462
## Start of the optimization  2  :
## The number of iterations is  31
##  The value of the  marginal posterior  function is  -127.7353
##  Optimized range parameters are 0.1932036 61631867 65398881 709.2223 31879.86 806.2717 3478.621 33790
##  Optimized nugget parameter is 0
##  Convergence:  TRUE
```

```r
m.subset <- rgasp(design = input[ ,c(1,4,6,7,8)], response = output,
                  nugget.est=TRUE)
```

```
## The upper bounds of the range parameters are 19.34479 23423.98 22990.27 107444.3 416986.6 Inf
## The initial values of range parameters are 0.3868958 468.4796 459.8055 2148.887 8339.732
## Start of the optimization  1  :
## The number of iterations is  45
##  The value of the  marginal posterior  function is  -126.1572
##  Optimized range parameters are 0.1836021 693.1441 777.1468 3167.149 33145.77
##  Optimized nugget parameter is 0.00000000000003253473
##  Convergence:  TRUE
## The initial values of range parameters are 0.2240224 271.2614 266.2389 1244.26 4828.915
## Start of the optimization  2  :
## The number of iterations is  45
##  The value of the  marginal posterior  function is  -126.1572
##  Optimized range parameters are 0.1836022 693.1457 777.1462 3167.157 33145.8
##  Optimized nugget parameter is 0.00000000000005056662
##  Convergence:  TRUE
```

```r
dk.full <- km(design = input, response = output)
```

```
##
## optimisation start
## ------------------
## * estimation method   : MLE
## * optimisation method : BFGS
## * analytical gradient : used
## * trend model : ~1
## * covariance model :
##   - type :  matern5_2
##   - nugget : NO
##   - parameters lower bounds :  0.0000000001 0.0000000001 0.0000000001 0.0000000001 0.0000000001 0.000
##   - parameters upper bounds :  0.1959501 97637.49 103605.2 237.2696 103.556 232.8764 1088.341 4223.80
##   - best initial criterion value(s) :  -175.6162
##
## N = 8, M = 5 machine precision = 2.22045e-16
## At X0, 0 variables are exactly at the bounds
## At iterate     0  f=       175.62  |proj g|=      0.16924
## At iterate     1  f =        172.3  |proj g|=      0.073955
## At iterate     2  f =       171.83  |proj g|=      0.065994
## At iterate     3  f =       171.66  |proj g|=       0.14412
## At iterate     4  f =        171.6  |proj g|=      0.057194
## At iterate     5  f =        171.6  |proj g|=      0.056321
## At iterate     6  f =        171.6  |proj g|=      0.051184
## At iterate     7  f =        171.6  |proj g|=      0.056248
## At iterate     8  f =        171.6  |proj g|=      0.056325
## At iterate     9  f =        171.6  |proj g|=      0.056448
## At iterate    10  f =        171.6  |proj g|=      0.056649
## At iterate    11  f =        171.6  |proj g|=      0.056975
## At iterate    12  f =        171.6  |proj g|=      0.057508
## At iterate    13  f =       171.59  |proj g|=      0.058376
## At iterate    14  f =       171.57  |proj g|=      0.059816
```

```
## At iterate    15  f =       171.51  |proj g|=      0.062152
## At iterate    16  f =       171.36  |proj g|=      0.065673
## At iterate    17  f =       171.11  |proj g|=      0.067085
## At iterate    18  f =        170.9  |proj g|=      0.060556
## At iterate    19  f =       170.87  |proj g|=       0.13976
## At iterate    20  f =       170.87  |proj g|=      0.057084
## At iterate    21  f =       170.87  |proj g|=     0.0085229
## At iterate    22  f =       170.87  |proj g|=     0.0085225
##
## iterations 22
## function evaluations 26
## segments explored during Cauchy searches 23
## BFGS updates skipped 0
## active bounds at final generalized Cauchy point 1
## norm of the final projected gradient 0.00852255
## final function value 170.867
##
## F = 170.867
## final  value 170.866677
## converged
```

```
dk.subset <- km(design = input[ ,c(1,4,6,7,8)], response = output,
                nugget.estim=TRUE)
```

```
##
## optimisation start
## ------------------
## * estimation method   : MLE
## * optimisation method : BFGS
## * analytical gradient : used
## * trend model : ~1
## * covariance model :
##   - type :  matern5_2
##   - nugget : unknown homogenous nugget effect
##   - parameters lower bounds :  0.0000000001 0.0000000001 0.0000000001 0.0000000001 0.0000000001
##   - parameters upper bounds :  0.1959501 237.2696 232.8764 1088.341 4223.802
##   - upper bound for alpha   :  1
##   - best initial criterion value(s) :  -200.1697
##
## N = 6, M = 5 machine precision = 2.22045e-16
## At X0, 0 variables are exactly at the bounds
## At iterate     0  f=       200.17  |proj g|=      0.84835
## At iterate     1  f =       175.57  |proj g|=       0.31708
## At iterate     2  f =       145.53  |proj g|=       0.15171
## ys=-2.211e+02  -gs= 1.281e+01, BFGS update SKIPPED
## At iterate     3  f =       142.16  |proj g|=      0.094111
## At iterate     4  f =       141.94  |proj g|=      0.087472
## At iterate     5  f =       141.84  |proj g|=       0.11877
## At iterate     6  f =       141.83  |proj g|=      0.079792
## At iterate     7  f =       141.83  |proj g|=      0.051442
## At iterate     8  f =       141.83  |proj g|=      0.051438
## At iterate     9  f =       141.83  |proj g|=      0.051438
##
## iterations 9
```

```
## function evaluations 13
## segments explored during Cauchy searches 11
## BFGS updates skipped 1
## active bounds at final generalized Cauchy point 1
## norm of the final projected gradient 0.0514375
## final function value 141.829
##
## F = 141.829
## final  value 141.828536
## converged
```

Noisy outputs